Mini project report on

CARPOOLING COORDINATION WITH ROUTE OPTIMIZATION

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS351A – DBMS Project

Submitted by:

Piyush A Patel	**PES2UG24CS350**
Ritesh Paruchuri	**PES2UG24CS340**

under the guidance of

Prof. Sheela Devi

Assistant Professor

PES University

AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Disasters data Management System

is a bonafide work carried out by

Piyush A Patel	**PES2UG24CS350**
Ritesh Paruchuri	**PES2UG24CS340**

In partial fulfilment for the completion of fifth semester DBMS Project (UE24CS351A) in the Program of Study -Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature

Prof. Sheela Devi

Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Disasters Data Management System** has been carried out by us under the guidance of **Prof. Sheela Devi, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.

Piyush A Patel	**PES2UG24CS350**	**<Signature>**
Ritesh Paruchuri	**PES2UG24CS340**	**<Signature>**

ACKNOWLEDGEMENT

I would like to express my gratitude to Prof. Sheela Devi, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS351A - DBMS Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

Carpooling Coordination with Route Optimization, implemented as CarpoolConnect, is a web-based application designed to coordinate shared rides between drivers and passengers. The system allows users to register and authenticate, register vehicles, publish rides, search for available rides, and book seats. A driver account can also act as a passenger when booking another driver's ride.

The application uses Flask for backend services, PostgreSQL for relational data storage, and pgvector for vector-based candidate ranking during nearby route matching. Ride routes are represented using ordered geographic stops and a 16-dimensional route embedding. Nearby matching combines vector similarity with geographic route-insertion and detour validation. Booking operations use database transactions and row-level locking to protect seat availability when concurrent booking requests occur.

The system also provides separate views for a user's own bookings and bookings made by passengers on rides owned by a driver. PostgreSQL trigger-based notifications inform riders and drivers about booking and cancellation events. The project demonstrates relational database design, primary and foreign key constraints, indexing, parameterized queries, transactions, row-level locking, vector indexing, PL/pgSQL functions, and triggers.

Keywords: Carpooling, Flask, PostgreSQL, pgvector, route matching, transactions, database triggers, ride booking.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	11
2.	PROBLEM DEFINITION	12
3.	ER MODEL	13
4.	ER TO RELATIONAL MAPPING	14
5.	DDL STATEMENTS	15
6.	DML STATEMENTS	20
7.	QUERIES (SIMPLE QUERY AND UPDATE AND DELETE OPERATION, CORRELATED QUERY AND NESTED QUERY)	23
8.	STORED PROCEDURE, FUNCTIONS AND TRIGGERS	30
9.	FRONT END DEVELOPMENT	40
REFERENCES/BIBLIOGRAPHY		41
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		42

LIST OF FIGURES

Figure	Description
Figure 3.1	Entity–Relationship diagram
Figure 4.1	Relational schema diagram
Figure 5.1	DDL execution and table creation evidence
Figure 6.1	Table records and DML evidence
Figure 7.1	Aggregate query output
Figure 7.2	UPDATE operation output
Figure 7.3	DELETE operation output
Figure 7.4	Correlated query output
Figure 7.5	Nested query output
Figure 8.1	Notification function definition
Figure 8.2	Notification trigger definition
Figure 8.3	Notification execution output

1. INTRODUCTION

Transportation demand, rising travel costs, and underutilized vehicle capacity create an opportunity for coordinated ride sharing. CarpoolConnect provides a shared platform where drivers can publish planned journeys and passengers can discover rides that align with their travel requirements.

Unlike a simple listing application, the project includes route-aware matching. A passenger can search for rides with the same source and destination or submit nearby pickup and drop-off coordinates. The backend ranks candidate routes using pgvector and then checks whether the passenger's journey can be inserted into the existing stop sequence within an acceptable detour.

1.1 Objectives

- Provide account registration, login, logout, and session-based access control.
- Allow drivers to register vehicles and publish rides with departure time, available seats, and price.
- Support exact and nearby route matching.
- Support booking and cancellation while maintaining correct seat availability.
- Provide separate My Bookings and Passenger Bookings views.
- Generate booking-event notifications using a PostgreSQL trigger.
- Demonstrate DBMS concepts including relational constraints, indexes, transactions, row-level locking, functions, and triggers.

1.2 Technology Stack

Layer	Technology / Component
Frontend	HTML, CSS, JavaScript, Jinja templates
Backend	Python, Flask
Database	PostgreSQL
Vector search	pgvector extension
Database driver	Psycopg 3
Map interface	Leaflet with OpenStreetMap tiles
Geocoding	Nominatim integration

1.3 Scope

The documented scope includes authentication, vehicle and ride management, exact and nearby ride matching, booking, cancellation, driver/passenger booking views, and notifications. A ratings feature is excluded from the functional scope of this report. The supplied legacy ER/DDL artifacts still contain a RATINGS entity/table; these artifacts must be revised before final submission to make the project scope consistent.

2. PROBLEM DEFINITION

Conventional ride coordination often relies on informal communication or static ride listings. Such approaches make it difficult to identify rides that pass sufficiently close to a passenger's pickup and drop-off points. They can also create booking inconsistencies when multiple passengers attempt to reserve the remaining seats at the same time.

2.1 Problem Statement

To design and implement a database-driven carpooling coordination system that allows drivers to publish rides and passengers to discover and reserve suitable journeys, while maintaining data integrity, controlling seat availability under concurrent requests, and supporting route-aware matching.

2.2 Functional Requirements

Users can register, log in, and log out.

Drivers can register vehicles and publish rides.

Users can search rides by source and destination.

Users can search nearby routes using pickup and drop-off coordinates.

Users can book seats and cancel their own confirmed bookings.

Drivers can view passenger bookings and manage eligible rides.

Users can view and mark notifications as read.

2.3 Non-functional Requirements

Data integrity through primary keys, foreign keys, unique constraints, and CHECK constraints.

Secure password storage using password hashing.

Parameterized SQL for user-supplied values.

Transactional booking operations with row-level locking.

Maintainable separation between page routes, API routes, database access, and frontend assets.

3. ER MODEL

The ER model identifies the principal entities involved in carpool coordination: USER, VEHICLE, RIDE, STOP, and BOOKING. A user can own vehicles and offer rides. A ride uses a vehicle, contains ordered stops, and can receive multiple bookings. A booking belongs to a rider and references pickup and drop-off stops.

The originally supplied diagram also contains a RATINGS entity. As requested, ratings are excluded from the functional scope described in this report; however, the source diagram has not yet been redrawn to remove that legacy entity.

ER Diagram:

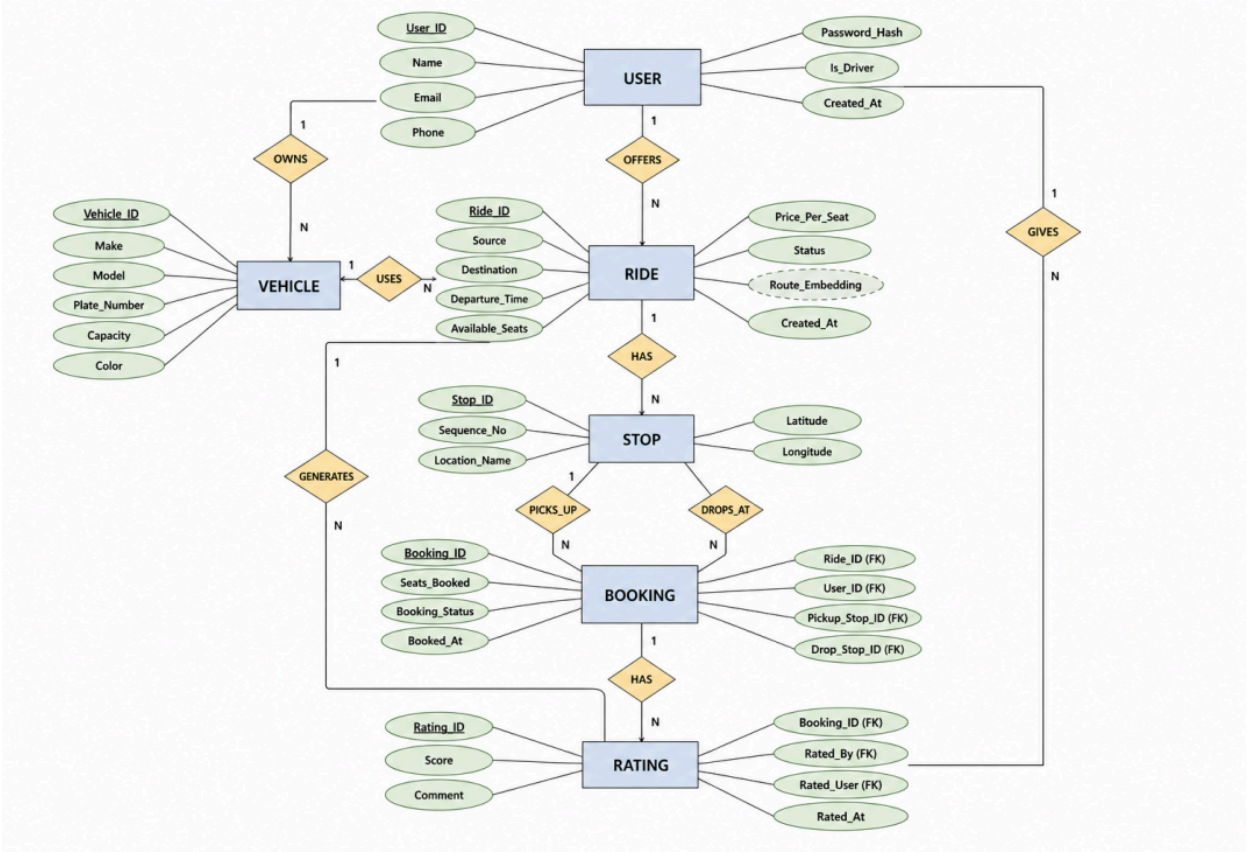


Figure 3.1: ER diagram supplied with the project materials.

3.1 Main Entities

Entity	Purpose
USER	Stores account identity, contact information, password hash, and driver flag.
VEHICLE	Stores vehicles registered by drivers.
RIDE	Stores ride details, driver, vehicle, departure time, seat availability, price, status, and route embedding.
STOP	Stores ordered route locations with latitude and longitude.
BOOKING	Stores passenger reservations, selected pickup/drop stops, seat count, status, and booking timestamp.
NOTIFICATIONS	Stores user-specific booking event notifications in the development database; migration needs to be added to the supplied SQL package.

4. ER TO RELATIONAL MAPPING

The ER model is mapped to relational tables using primary keys and foreign keys. One-to-many relationships are represented by placing the foreign key on the many-side relation. The route is represented as a RIDE row related to multiple STOP rows. BOOKING references the ride, rider, pickup stop, and drop-off stop.

4.1 STEPS OF ALGORITHM FOR CHOSEN PROBLEM

Create the USER relation with a unique user identifier and unique email and phone values.

Create VEHICLE and reference its owner through driver_id.

Create RIDE and reference both its driver and vehicle.

Create STOP and reference its parent ride; use sequence_no to preserve route order.

Create BOOKING and reference the ride, rider, pickup stop, and drop-off stop.

Define primary keys, foreign keys, uniqueness rules, CHECK constraints, and default values.

Create indexes for common joins, filters, and vector similarity search.

Represent ride routes using a fixed-dimension pgvector embedding for nearby candidate ranking.

4.2 COMPLETE DIAGRAM OF RELATIONAL MAPPING

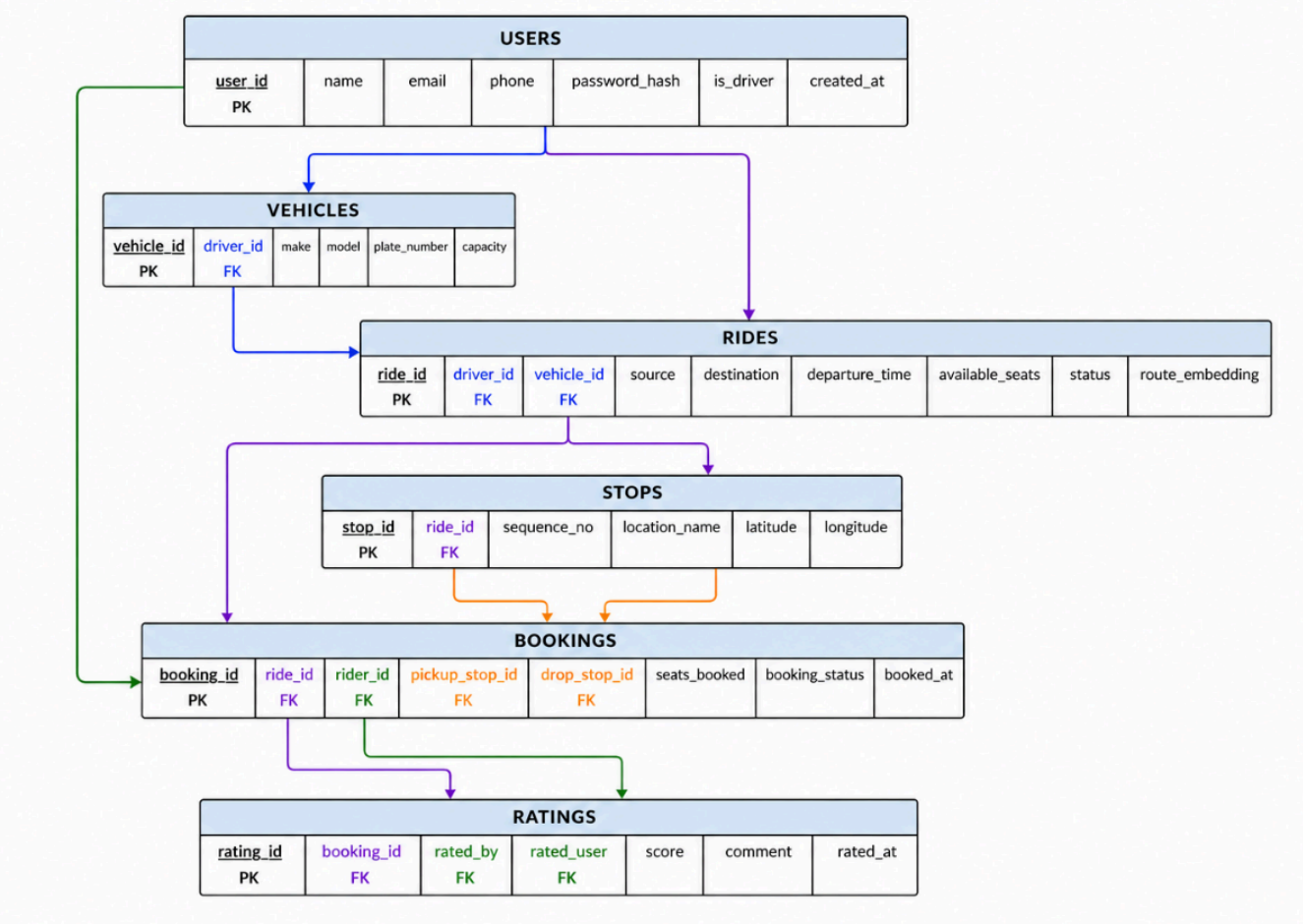


Figure 4.1: Relational schema diagram supplied with the project materials.

The supplied relational diagram includes USERS, VEHICLES, RIDES, STOPS, BOOKINGS, and RATINGS. The ratings relation is a legacy artifact and should be removed from the diagram and schema if ratings remain outside the final project scope.

5. DDL STATEMENTS

Data Definition Language (DDL) statements define the database structure, including tables, columns, primary keys, foreign keys, constraints, indexes, extensions, functions, and triggers. The project schema is implemented in PostgreSQL and uses pgvector for route embeddings.

The supplied DDL execution evidence shows creation of the carpool database, vector extension, core tables, indexes, function, and trigger. The table structure screenshots show USERS, VEHICLES, RIDES, STOPS, BOOKINGS, and RATINGS.

DDL Query Screenshots:

```
piyush-a-patel@piyush:~/Windows/Piyush/Projects/Python/Carpooling Coordination with Route Optimization$ sudo -u postgres psql -f TEAM_03_DDL.sql
psql:TEAM_03_DDL.sql:1: NOTICE:  database "carpool" does not exist, skipping
DROP DATABASE
CREATE DATABASE
You are now connected to database "carpool" as user "postgres".
CREATE EXTENSION
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
psql:TEAM_03_DDL.sql:91: NOTICE:  ivfflat index created with little data
DETAIL:  This will cause low recall.
HINT:  Drop the index until the table has more data.
CREATE INDEX
CREATE FUNCTION
CREATE TRIGGER
```

List of Tables:

```
piyush-a-patel@piyush:~/Windows/Piyush/Projects/Python/Carpooling Coordination with Route Optimization$ sudo -u postgres psql -d carpool
psql (16.15 (Ubuntu 16.15-1.pgdg24.04+2))
Type "help" for help.

carpool=# \dt
          List of relations
 Schema |   Name   | Type  |  Owner
--------+----------+-------+----------
 public | bookings | table | postgres
 public | ratings  | table | postgres
 public | rides    | table | postgres
 public | stops    | table | postgres
 public | users    | table | postgres
 public | vehicles | table | postgres
(6 rows)
```

Figure 5.1: DDL and table-definition evidence

Table 1 (USERS):

```
carpool=# \d users
                                      Table "public.users"
    Column     |            Type             | Collation | Nullable |               Default
---------------+-----------------------------+-----------+----------+--------------------------------------
 user_id       | integer                     |           | not null | nextval('users_user_id_seq'::regclass)
 name          | character varying(100)      |           | not null |
 email         | character varying(120)      |           | not null |
 phone         | character varying(15)       |           | not null |
 password_hash | character varying(255)      |           | not null |
 is_driver     | boolean                     |           |          | false
 created_at    | timestamp without time zone |           |          | CURRENT_TIMESTAMP
Indexes:
    "users_pkey" PRIMARY KEY, btree (user_id)
    "users_email_key" UNIQUE CONSTRAINT, btree (email)
    "users_phone_key" UNIQUE CONSTRAINT, btree (phone)
Referenced by:
    TABLE "bookings" CONSTRAINT "bookings_rider_id_fkey" FOREIGN KEY (rider_id) REFERENCES users(user_id) ON DELETE CASCADE
    TABLE "ratings" CONSTRAINT "ratings_rated_by_fkey" FOREIGN KEY (rated_by) REFERENCES users(user_id)
    TABLE "ratings" CONSTRAINT "ratings_rated_user_fkey" FOREIGN KEY (rated_user) REFERENCES users(user_id)
    TABLE "rides" CONSTRAINT "rides_driver_id_fkey" FOREIGN KEY (driver_id) REFERENCES users(user_id) ON DELETE CASCADE
    TABLE "vehicles" CONSTRAINT "vehicles_driver_id_fkey" FOREIGN KEY (driver_id) REFERENCES users(user_id) ON DELETE CASCADE
```

Table 2 (VEHICLES):

```
carpool=# \d VEHICLES
                                    Table "public.vehicles"
    Column    |         Type          | Collation | Nullable |                  Default
--------------+-----------------------+-----------+----------+-------------------------------------------
 vehicle_id   | integer               |           | not null | nextval('vehicles_vehicle_id_seq'::regclass)
 driver_id    | integer               |           | not null |
 make         | character varying(50) |           | not null |
 model        | character varying(50) |           | not null |
 plate_number | character varying(15) |           | not null |
 capacity     | smallint              |           | not null |
 color        | character varying(30) |           |          |
Indexes:
    "vehicles_pkey" PRIMARY KEY, btree (vehicle_id)
    "vehicles_plate_number_key" UNIQUE CONSTRAINT, btree (plate_number)
Check constraints:
    "vehicles_capacity_check" CHECK (capacity >= 1 AND capacity <= 8)
Foreign-key constraints:
    "vehicles_driver_id_fkey" FOREIGN KEY (driver_id) REFERENCES users(user_id) ON DELETE CASCADE
Referenced by:
    TABLE "rides" CONSTRAINT "rides_vehicle_id_fkey" FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id) ON DELETE CASCADE
```

Figure 5.2: DDL and table-definition evidence

Table 3 (RIDES):

```
carpool=# \d rides
                                        Table "public.rides"
     Column      |           Type           | Collation | Nullable |              Default
-----------------+--------------------------+-----------+----------+-----------------------------------
 ride_id         | integer                  |           | not null | nextval('rides_ride_id_seq'::regclass)
 driver_id       | integer                  |           | not null |
 vehicle_id      | integer                  |           | not null |
 source          | character varying(150)   |           | not null |
 destination     | character varying(150)   |           | not null |
 departure_time  | timestamp without time zone |        | not null |
 available_seats | smallint                 |           | not null |
 price_per_seat  | numeric(8,2)             |           | not null |
 status          | character varying(20)    |           |          | 'scheduled'::character varying
 route_embedding | vector(16)               |           |          |
 created_at      | timestamp without time zone |        |          | CURRENT_TIMESTAMP
Indexes:
    "rides_pkey" PRIMARY KEY, btree (ride_id)
    "idx_rides_driver" btree (driver_id)
    "idx_rides_route_embedding" ivfflat (route_embedding) WITH (lists='50')
    "idx_rides_status" btree (status)
Check constraints:
    "rides_available_seats_check" CHECK (available_seats >= 0)
    "rides_price_per_seat_check" CHECK (price_per_seat >= 0::numeric)
    "rides_status_check" CHECK (status::text = ANY (ARRAY['scheduled'::character varying, 'ongoing'::character varying, 'completed'::character varying, 'cancelled'::character varying]::text[
]))
Foreign-key constraints:
    "rides_driver_id_fkey" FOREIGN KEY (driver_id) REFERENCES users(user_id) ON DELETE CASCADE
    "rides_vehicle_id_fkey" FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id) ON DELETE CASCADE
Referenced by:
    TABLE "bookings" CONSTRAINT "bookings_ride_id_fkey" FOREIGN KEY (ride_id) REFERENCES rides(ride_id) ON DELETE CASCADE
    TABLE "stops" CONSTRAINT "stops_ride_id_fkey" FOREIGN KEY (ride_id) REFERENCES rides(ride_id) ON DELETE CASCADE
```

Table 4 (STOPS):

```
carpool=# \d stops
                                        Table "public.stops"
    Column     |          Type          | Collation | Nullable |               Default
---------------+------------------------+-----------+----------+--------------------------------------
 stop_id       | integer                |           | not null | nextval('stops_stop_id_seq'::regclass)
 ride_id       | integer                |           | not null |
 sequence_no   | smallint               |           | not null |
 location_name | character varying(150) |           | not null |
 latitude      | numeric(9,6)           |           | not null |
 longitude     | numeric(9,6)           |           | not null |
Indexes:
    "stops_pkey" PRIMARY KEY, btree (stop_id)
    "idx_stops_ride" btree (ride_id)
    "stops_ride_id_sequence_no_key" UNIQUE CONSTRAINT, btree (ride_id, sequence_no)
Foreign-key constraints:
    "stops_ride_id_fkey" FOREIGN KEY (ride_id) REFERENCES rides(ride_id) ON DELETE CASCADE
Referenced by:
    TABLE "bookings" CONSTRAINT "bookings_drop_stop_id_fkey" FOREIGN KEY (drop_stop_id) REFERENCES stops(stop_id)
    TABLE "bookings" CONSTRAINT "bookings_pickup_stop_id_fkey" FOREIGN KEY (pickup_stop_id) REFERENCES stops(stop_id)

carpool=#
```

Figure 5.3: DDL and table-definition evidence

Table 5 (BOOKINGS):

```
carpool=# \d bookings
                                        Table "public.bookings"
     Column     |           Type           | Collation | Nullable |                  Default
----------------+--------------------------+-----------+----------+-------------------------------------------
 booking_id     | integer                  |           | not null | nextval('bookings_booking_id_seq'::regclass)
 ride_id        | integer                  |           | not null |
 rider_id       | integer                  |           | not null |
 pickup_stop_id | integer                  |           | not null |
 drop_stop_id   | integer                  |           | not null |
 seats_booked   | smallint                 |           | not null |
 booking_status | character varying(20)    |           |          | 'pending'::character varying
 booked_at      | timestamp without time zone |        |          | CURRENT_TIMESTAMP
Indexes:
    "bookings_pkey" PRIMARY KEY, btree (booking_id)
    "idx_bookings_ride" btree (ride_id)
    "idx_bookings_rider" btree (rider_id)
Check constraints:
    "bookings_booking_status_check" CHECK (booking_status::text = ANY (ARRAY['pending'::character varying, 'confirmed'::character varying, 'cancelled'::character varying, 'completed'::charac
ter varying]::text[]))
    "bookings_check" CHECK (pickup_stop_id <> drop_stop_id)
    "bookings_seats_booked_check" CHECK (seats_booked > 0)
Foreign-key constraints:
    "bookings_drop_stop_id_fkey" FOREIGN KEY (drop_stop_id) REFERENCES stops(stop_id)
    "bookings_pickup_stop_id_fkey" FOREIGN KEY (pickup_stop_id) REFERENCES stops(stop_id)
    "bookings_ride_id_fkey" FOREIGN KEY (ride_id) REFERENCES rides(ride_id) ON DELETE CASCADE
    "bookings_rider_id_fkey" FOREIGN KEY (rider_id) REFERENCES users(user_id) ON DELETE CASCADE
Referenced by:
    TABLE "ratings" CONSTRAINT "ratings_booking_id_fkey" FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE CASCADE
Triggers:
    trg_booking_seats AFTER INSERT ON bookings FOR EACH ROW EXECUTE FUNCTION fn_update_seats()
```

Table 6 (RATINGS):

```
carpool=# \d ratings
                                      Table "public.ratings"
   Column    |            Type             | Collation | Nullable |                  Default
-------------+-----------------------------+-----------+----------+-------------------------------------------
 rating_id   | integer                     |           | not null | nextval('ratings_rating_id_seq'::regclass)
 booking_id  | integer                     |           | not null |
 rated_by    | integer                     |           | not null |
 rated_user  | integer                     |           | not null |
 score       | smallint                    |           | not null |
 comment     | character varying(300)      |           |          |
 rated_at    | timestamp without time zone |           |          | CURRENT_TIMESTAMP
Indexes:
    "ratings_pkey" PRIMARY KEY, btree (rating_id)
    "ratings_booking_id_key" UNIQUE CONSTRAINT, btree (booking_id)
Check constraints:
    "ratings_check" CHECK (rated_by <> rated_user)
    "ratings_score_check" CHECK (score >= 1 AND score <= 5)
Foreign-key constraints:
    "ratings_booking_id_fkey" FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE CASCADE
    "ratings_rated_by_fkey" FOREIGN KEY (rated_by) REFERENCES users(user_id)
    "ratings_rated_user_fkey" FOREIGN KEY (rated_user) REFERENCES users(user_id)
```

Figure 5.4: DDL and table-definition evidence

5.1 Important database design notes

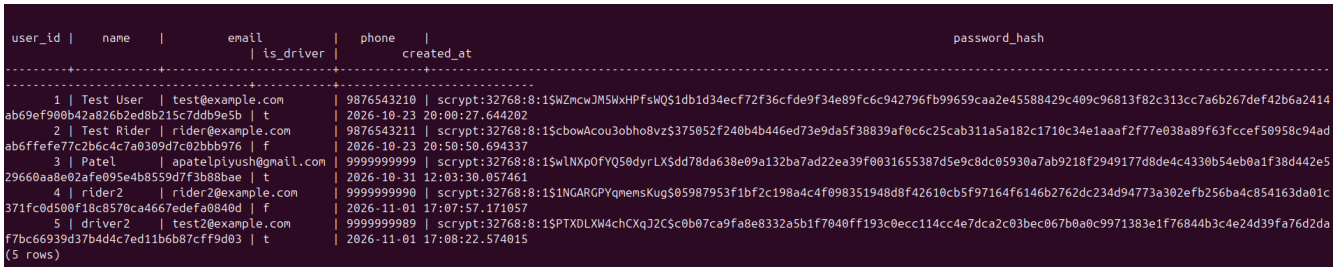
- Foreign keys maintain relationships between users, vehicles, rides, stops, and bookings.
- CHECK constraints validate values such as vehicle capacity, non-negative available seats, positive seats booked, price, and allowed status values.
- Unique constraints prevent duplicate account email/phone values, duplicate vehicle plate numbers, and duplicate stop sequence numbers within a ride.
- The route embedding column uses vector(16), and an IVFFlat index is defined for similarity search.

Implementation reconciliation: the supplied SQL package contains a legacy booking seat-decrement trigger, while the current Flask booking workflow decrements seats within a Python-managed transaction. Both mechanisms must not be active together because that would decrement seat availability twice. The fresh-install SQL should omit the legacy seat trigger or the application logic must be redesigned consistently.

The current application also expects pickup_stop_inserted and drop_stop_inserted columns in BOOKINGS and a NOTIFICATIONS table. These are present in the development database but are not fully represented in the supplied DDL package. A consolidated migration is required for reproducible installation.

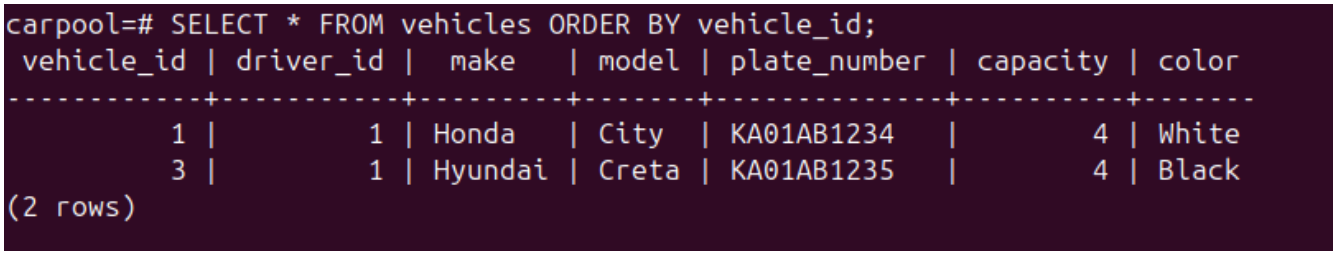
6. DML STATEMENTS

Data Manipulation Language (DML) statements insert, retrieve, update, and delete records. The screenshots in this chapter show existing records from the development database and safe demonstration transactions for INSERT.



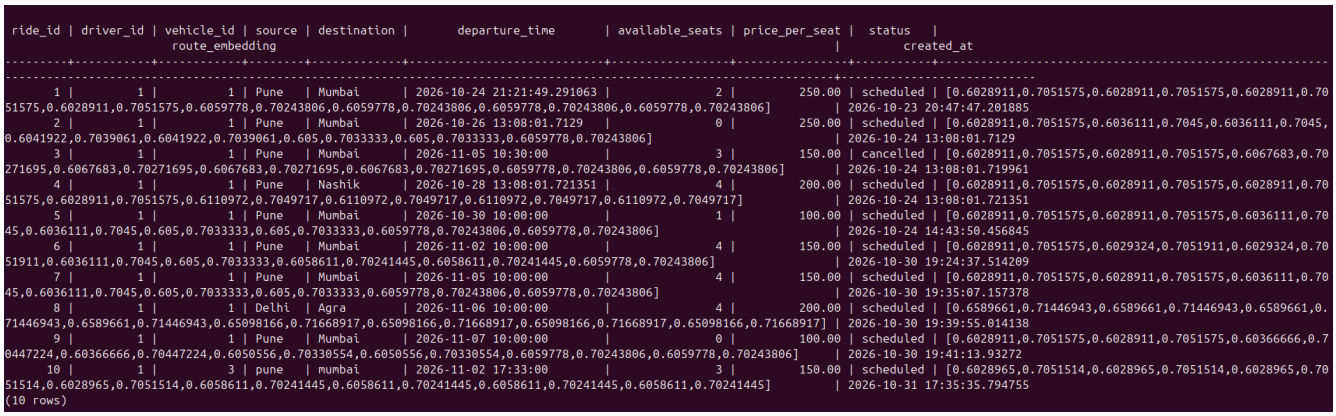
```
 user_id |    name    |         email          |   phone    |                                                                  password_hash
                                                                  | is_driver |         created_at
---------+------------+------------------------+------------+----------------------------------------------------------------------------------------------------------------------------------------------------------
----------------------------------+----------+----------------------------
       1 | Test User  | test@example.com       | 9876543210 | scrypt:32768:8:1$WZmcwJM5WxHPfsWQ$1db1d34ecf72f36cfde9f34e89fc6c942796fb99659caa2e45588429c409c96813f82c313cc7a6b267def42b6a2414
ab69ef900b42a826b2ed8b215c7ddb9e5b | t        | 2026-10-23 20:00:27.644202
       2 | Test Rider | rider@example.com      | 9876543211 | scrypt:32768:8:1$cbowAcou3obho8vz$375052f240b4b446ed73e9da5f38839af0c6c25cab311a5a182c1710c34e1aaaf2f77e038a89f63fccef50958c94ad
ab6ffefe77c2b6c4c7a0309d7c02bbb976 | f        | 2026-10-23 20:50:50.694337
       3 | Patel      | apatelpiyush@gmail.com | 9999999999 | scrypt:32768:8:1$wlNXpOfYQ50dyrLX$dd78da638e09a132ba7ad22ea39f0031655387d5e9c8dc05930a7ab9218f2949177d8de4c4330b54eb0a1f38d442e5
29660aa8e02afe095e4b8559d7f3b88bae | t        | 2026-10-31 12:03:30.057461
       4 | rider2     | rider2@example.com     | 9999999990 | scrypt:32768:8:1$1NGARGPYqmemsKug$05987953f1bf2c198a4c4f098351948d8f42610cb5f97164f6146b2762dc234d94773a302efb256ba4c854163da01c
371fc0d500f18c8570ca4667edefa0840d | f        | 2026-11-01 17:07:57.171057
       5 | driver2    | test2@example.com      | 9999999989 | scrypt:32768:8:1$PTXDLXW4chCXqJ2C$c0b07ca9fa8e8332a5b1f7040ff193c0ecc114cc4e7dca2c03bec067b0a0c9971383e1f76844b3c4e24d39fa76d2da
f7bc66939d37b4d4c7ed11b6b87cff9d03 | t        | 2026-11-01 17:08:22.574015
(5 rows)
```

Figure 6.1: Table contents / DML evidence from the supplied screenshot set.



```
carpool=# SELECT * FROM vehicles ORDER BY vehicle_id;
 vehicle_id | driver_id |  make   | model | plate_number | capacity | color
------------+-----------+---------+-------+--------------+----------+-------
          1 |         1 | Honda   | City  | KA01AB1234   |        4 | White
          3 |         1 | Hyundai | Creta | KA01AB1235   |        4 | Black
(2 rows)
```

Figure 6.2: Table contents / DML evidence from the supplied screenshot set.



```
 ride_id | driver_id | vehicle_id | source | destination |      departure_time      | available_seats | price_per_seat |  status   |
                    route_embedding                                                                                                                    |         created_at
---------+-----------+------------+--------+-------------+--------------------------+-----------------+----------------+-----------+----------------------------------------------------------------------------------------------------------------------------------------------------
----------------------------------------------------------------------------------------------------------------------------------------------------+----------------------------
       1 |         1 |          1 | Pune   | Mumbai      | 2026-10-24 21:21:49.291063 |               2 |         250.00 | scheduled | [0.6028911,0.7051575,0.6028911,0.7051575,0.6028911,0.70
51575,0.6028911,0.7051575,0.6059778,0.70243806,0.6059778,0.70243806,0.6059778,0.70243806,0.6059778,0.70243806]                                                                                                                
```

```
carpool=# SELECT * FROM rides ORDER BY ride_id;
carpool=# SELECT * FROM stops ORDER BY ride_id, sequence_no;
 stop_id | ride_id | sequence_no |                                                                location_name                                                                | latitude  | longitude
---------+---------+-------------+---------------------------------------------------------------------------------------------------------------------------------------------+-----------+-----------
       1 |       1 |           1 | Pune                                                                                                                                        | 18.520400 | 73.856700
       2 |       1 |           2 | Mumbai                                                                                                                                      | 19.076000 | 72.877700
       3 |       2 |           1 | Pune                                                                                                                                        | 18.520400 | 73.856700
      11 |       2 |           2 | Test Pickup                                                                                                                                 | 18.650000 | 73.620000
       4 |       2 |           3 | Lonavala                                                                                                                                    | 18.754600 | 73.406200
      12 |       2 |           4 | Test Drop                                                                                                                                   | 18.900000 | 73.200000
       5 |       2 |           5 | Mumbai                                                                                                                                      | 19.076000 | 72.877700
       6 |       3 |           1 | Pune                                                                                                                                        | 18.520400 | 73.856700
       7 |       3 |           2 | Thane                                                                                                                                       | 19.218300 | 72.978100
       8 |       3 |           3 | Mumbai                                                                                                                                      | 19.076000 | 72.877700
       9 |       4 |           1 | Pune                                                                                                                                        | 18.520400 | 73.856700
      10 |       4 |           2 | Nashik                                                                                                                                      | 19.997500 | 73.789800
      13 |       5 |           1 | Pune                                                                                                                                        | 18.520400 | 73.856700
      15 |       5 |           2 | Test Pickup                                                                                                                                 | 18.650000 | 73.620000
      16 |       5 |           3 | Test Drop                                                                                                                                   | 18.900000 | 73.200000
      14 |       5 |           4 | Mumbai                                                                                                                                      | 19.076000 | 72.877700
      18 |       6 |           1 | Pune                                                                                                                                        | 18.520400 | 73.856700
      34 |       6 |           2 | Pune Railway Station, Somwar Peth, Pune, Pune City Subdistrict, Pune District, Maharashtra, 411001, India                                   | 18.527830 | 73.868777
      20 |       6 |           3 | Test Pickup 2                                                                                                                               | 18.650000 | 73.620000
      21 |       6 |           4 | Test Drop 2                                                                                                                                 | 18.900000 | 73.200000
      35 |       6 |           5 | Mumbai, Mumbai Suburban District, Maharashtra, 400051, India                                                                                | 19.054999 | 72.869204
      19 |       6 |           6 | Mumbai                                                                                                                                      | 19.076000 | 72.877700
      22 |       7 |           1 | Pune                                                                                                                                        | 18.520400 | 73.856700
      24 |       7 |           2 | Test Pickup 3                                                                                                                               | 18.650000 | 73.620000
      25 |       7 |           3 | Test Drop 3                                                                                                                                 | 18.900000 | 73.200000
      23 |       7 |           4 | Mumbai                                                                                                                                      | 19.076000 | 72.877700
      26 |       8 |           1 | Delhi                                                                                                                                       | 28.613900 | 77.209000
      27 |       8 |           2 | Agra                                                                                                                                        | 27.176700 | 78.008100
      28 |       9 |           1 | Pune                                                                                                                                        | 18.520400 | 73.856700
      30 |       9 |           2 | Concurrent Pickup B                                                                                                                         | 18.660000 | 73.610000
      31 |       9 |           3 | Concurrent Drop B                                                                                                                           | 18.910000 | 73.190000
      29 |       9 |           4 | Mumbai                                                                                                                                      | 19.076000 | 72.877700
      32 |      10 |           1 | pune                                                                                                                                        | 18.521374 | 73.854507
      33 |      10 |           2 | mumbai                                                                                                                                      | 19.054999 | 72.869204
(34 rows)
```

Figure 6.4: Table contents / DML evidence from the supplied screenshot set.

```
carpool=# SELECT * FROM bookings ORDER BY booking_id;
 booking_id | ride_id | rider_id | pickup_stop_id | drop_stop_id | seats_booked | booking_status |         booked_at          | pickup_stop_inserted | drop_stop_inserted
------------+---------+----------+----------------+--------------+--------------+----------------+----------------------------+----------------------+--------------------
          2 |       1 |        2 |              1 |            2 |            1 | confirmed      | 2026-10-23 21:05:12.944101 | f                    | f
          3 |       2 |        2 |             11 |           12 |            1 | cancelled      | 2026-10-24 14:24:48.001802 | f                    | f
          4 |       2 |        2 |             11 |           12 |            1 | confirmed      | 2026-10-24 14:41:37.999555 | f                    | f
         14 |       5 |        2 |             13 |           14 |            1 | cancelled      | 2026-10-29 17:07:41.122001 | f                    | f
         15 |       5 |        2 |             15 |           16 |            1 | cancelled      | 2026-10-29 17:22:47.909154 | f                    | f
         16 |       1 |        2 |              1 |            2 |            1 | cancelled      | 2026-10-29 17:34:40.495914 | f                    | f
         17 |       6 |        2 |             20 |           21 |            1 | cancelled      | 2026-10-30 19:26:08.547443 | t                    | t
         18 |       7 |        2 |             24 |           25 |            1 | cancelled      | 2026-10-30 19:38:02.923825 | t                    | t
         19 |       9 |        2 |             30 |           31 |            1 | confirmed      | 2026-10-30 19:41:36.521179 | t                    | t
         20 |       6 |        2 |             34 |           35 |            1 | cancelled      | 2026-11-01 15:10:45.607892 | t                    | t
         27 |       6 |        3 |             34 |           35 |            1 | cancelled      | 2026-11-01 16:49:05.349534 | f                    | f
(11 rows)
```

Figure 6.5: Table contents / DML evidence from the supplied screenshot set.

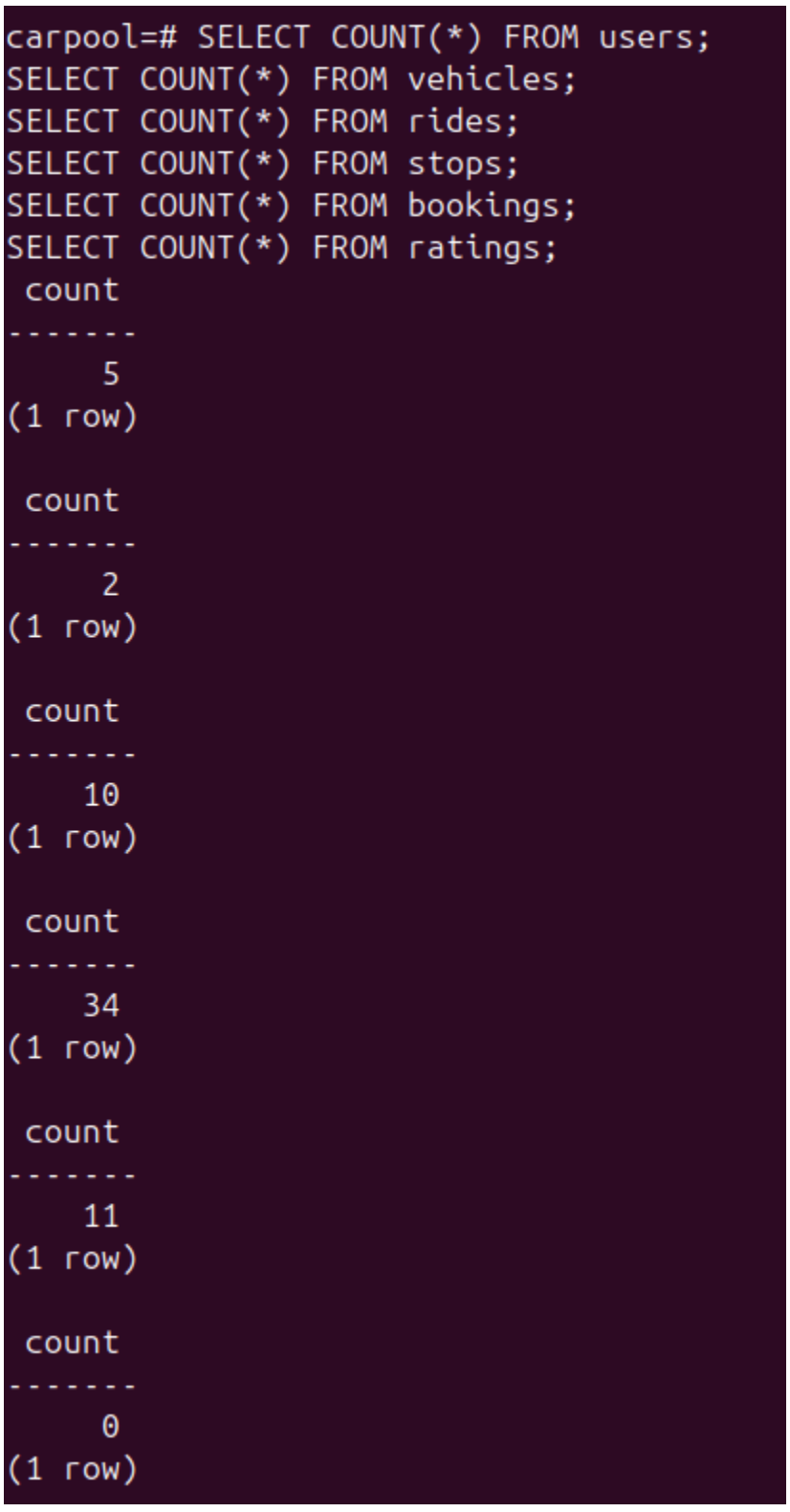


Figure 6.6: Table contents / DML evidence from the supplied screenshot set.

The record screenshots cover USERS, VEHICLES, RIDES, STOPS, and BOOKINGS. The original screenshot set also contains the legacy RATINGS table; it is not treated as a functional module in this report.

6.1 INSERT operation

```
BEGIN;
INSERT INTO users (name, email, phone, password_hash, is_driver)
VALUES ('DML Test User', 'dml_test_001@example.com',
      '9876501234', 'temporary_demo_hash', FALSE)
RETURNING *;
ROLLBACK;
```

A temporary test user was inserted inside a transaction and returned using RETURNING *. The transaction was rolled back after capturing the output, preserving the database contents.

```
carpool=# \pset pager off

SELECT pg_get_functiondef(
    'notify_booking_events()'::regprocedure
);
Pager usage is off.
\pset: extra argument "SELECT" ignored
\pset: extra argument "pg_get_functiondef(" ignored
\pset: extra argument "notify_booking_events()::regprocedure" ignored
\pset: extra argument ");" ignored
carpool=#
```

Figure 6.7: INSERT statement and returned test record; transaction rolled back.

7. QUERIES

7.1 SIMPLE QUERY WITH GROUP BY, AGRREGATE

```
SELECT u.user_id, u.name,
    COUNT(r.ride_id) AS total_rides,
    SUM(r.available_seats) AS total_available_seats,
    AVG(r.price_per_seat) AS average_price
```

FROM users u

JOIN rides r ON u.user_id = r.driver_id

GROUP BY u.user_id, u.name

ORDER BY total_rides DESC;

```
carpool=# SELECT
    u.user_id,
    u.name,
    COUNT(r.ride_id) AS total_rides,
    SUM(r.available_seats) AS total_available_seats,
    AVG(r.price_per_seat) AS average_price
FROM users u
JOIN rides r ON u.user_id = r.driver_id
GROUP BY u.user_id, u.name
ORDER BY total_rides DESC;
 user_id |   name    | total_rides | total_available_seats |    average_price
---------+-----------+-------------+-----------------------+----------------------
       1 | Test User |          10 |                    25 | 170.0000000000000000
(1 row)
```

Figure 7.1: GROUP BY and aggregate query output.

7.2 UPDATE OPERATION

BEGIN;

UPDATE rides

SET price_per_seat = price_per_seat + 10

WHERE ride_id = 6

RETURNING ride_id, price_per_seat;

ROLLBACK;

The statement temporarily increases the price per seat for ride 6 and returns the modified value. ROLLBACK restores the original value after the demonstration.

```
carpool=# BEGIN;

UPDATE rides
SET price_per_seat = price_per_seat + 10
WHERE ride_id = 6
RETURNING ride_id, price_per_seat;

ROLLBACK;
BEGIN
 ride_id | price_per_seat
---------+----------------
       6 |         160.00
(1 row)

UPDATE 1
ROLLBACK
carpool=#
```

Figure 7.2: UPDATE operation output; transaction rolled back.

7.3 DELETE OPERATION

```
BEGIN;
INSERT INTO users (name, email, phone, password_hash, is_driver)
VALUES ('Delete Test User', 'delete_test_001@example.com',
    '9876501235', 'temporary_demo_hash', FALSE)
RETURNING user_id;


DELETE FROM users
WHERE email = 'delete_test_001@example.com'
RETURNING *;
ROLLBACK;
```

A temporary user is inserted and deleted within the same transaction. The DELETE ... RETURNING clause displays the deleted row. ROLLBACK ensures no permanent data change.

```
carpool=# BEGIN;

INSERT INTO users (
    name,
    email,
    phone,
    password_hash,
    is_driver
)
VALUES (
    'DML Test User',
    'dml_test_001@example.com',
    '9876501234',
    'temporary_demo_hash',
    FALSE
)
RETURNING *;

ROLLBACK;
BEGIN
 user_id |     name      |          email           |   phone    |    password_hash    | is_driver |         created_at
---------+---------------+--------------------------+------------+---------------------+-----------+----------------------------
       6 | DML Test User | dml_test_001@example.com | 9876501234 | temporary_demo_hash | f         | 2026-11-01 17:51:31.738542
(1 row)

INSERT 0 1
ROLLBACK
carpool=#
```

Figure 7.3: DELETE operation output; transaction rolled back.

7.4 CORRELATED QUERY

```
SELECT u.user_id, u.name, u.email
FROM users u
WHERE EXISTS (
  SELECT 1
  FROM rides r
  WHERE r.driver_id = u.user_id
);
```

The subquery refers to u.user_id from the outer query. It therefore executes as a correlated existence test for each candidate user.

```
carpool=# SELECT
    u.user_id,
    u.name,
    u.email
FROM users u
WHERE EXISTS (
    SELECT 1
    FROM rides r
    WHERE r.driver_id = u.user_id
);
 user_id |   name    |      email
---------+-----------+------------------
       1 | Test User | test@example.com
(1 row)
```

Figure 7.4: Correlated query output.

7.5 NESTED QUERY

```
SELECT ride_id, source, destination, price_per_seat
FROM rides
WHERE price_per_seat > (
  SELECT AVG(price_per_seat)
  FROM rides
);
```

The inner query calculates the average price per seat. The outer query returns rides whose price is greater than that average.

```
carpool=# SELECT
    ride_id,
    source,
    destination,
    price_per_seat
FROM rides
WHERE price_per_seat > (
    SELECT AVG(price_per_seat)
    FROM rides
);
 ride_id | source | destination | price_per_seat
---------+--------+-------------+----------------
       2 | Pune   | Mumbai      |         250.00
       4 | Pune   | Nashik      |         200.00
       1 | Pune   | Mumbai      |         250.00
       8 | Delhi  | Agra        |         200.00
(4 rows)
```

Figure 7.5: Nested query output.

8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

PostgreSQL procedural functionality is used to generate notifications when booking events occur. A PL/pgSQL trigger function reads the driver associated with the booking's ride and inserts notifications for the rider and driver.

8.1 STORED PROCEDURES OR FUNCTIONS

The function notify_booking_events() is written in PL/pgSQL and is invoked by a row-level trigger on BOOKINGS. On insertion of a confirmed booking, it creates a booking-confirmation notification for the rider and a new-passenger notification for the driver. On cancellation of a previously confirmed booking, it creates a cancellation notification for the rider. The supplied screenshot shows the function through the rider cancellation insert; the remainder of the function is not visible in the evidence image.



```
                  pg_get_functiondef
-------------------------------------------------------------
 CREATE OR REPLACE FUNCTION public.notify_booking_events()+
  RETURNS trigger                                         +
  LANGUAGE plpgsql                                        +
 AS $function$                                            +
 DECLARE                                                  +
     driver_user_id INTEGER;                              +
 BEGIN                                                    +
     SELECT driver_id                                     +
     INTO driver_user_id                                  +
     FROM rides                                           +
     WHERE ride_id = NEW.ride_id;                         +
                                                          +
     -- New confirmed booking                             +
     IF TG_OP = 'INSERT'                                  +
        AND NEW.booking_status = 'confirmed' THEN         +
                                                          +
         INSERT INTO notifications (                      +
             user_id, booking_id, ride_id,                +
             title, message, notification_type            +
         )                                                +
         VALUES                                           +
         (                                                +
             NEW.rider_id,                                +
             NEW.booking_id,                              +
             NEW.ride_id,                                 +
             'Booking confirmed',                         +
             'Your ride booking has been confirmed.',     +
             'booking_confirmed'                          +
         ),                                               +
         (                                                +
             driver_user_id,                              +
             NEW.booking_id,                              +
             NEW.ride_id,                                 +
             'New passenger booking',                     +
             'A passenger has booked your ride.',         +
             'new_booking'                                +
         );                                               +
                                                          +
     -- Booking cancelled                                 +
     ELSIF TG_OP = 'UPDATE'                               +
           AND OLD.booking_status = 'confirmed'           +
           AND NEW.booking_status = 'cancelled' THEN      +
                                                          +
         INSERT INTO notifications (                      +
             user_id, booking_id, ride_id,                +
             title, message, notification_type            +
         )                                                +
         VALUES                                           +
         (                                                +
             NEW.rider_id,                                +
             NEW.booking_id,                              +
             NEW.ride_id,                                 +
             'Booking cancelled',                         +
             'Your booking has been cancelled.',          +
             'booking_cancelled'                          +
         ),                                               +
:
```

Figure 8.1: Available screenshot of notify_booking_events() function definition (lower portion not captured).

8.2 TRIGGERS

The trigger trg_booking_notifications is defined as an AFTER INSERT OR UPDATE OF booking_status row-level trigger on the bookings table. It executes notify_booking_events() so that notification records are generated as a consequence of booking state changes.

```
carpool=# SELECT
    trigger_name,
    event_manipulation,
    event_object_table,
    action_statement
FROM information_schema.triggers
WHERE trigger_name = 'trg_booking_notifications';
       trigger_name        | event_manipulation | event_object_table |             action_statement
---------------------------+--------------------+--------------------+-----------------------------------------
 trg_booking_notifications | INSERT             | bookings           | EXECUTE FUNCTION notify_booking_events()
 trg_booking_notifications | UPDATE             | bookings           | EXECUTE FUNCTION notify_booking_events()
(2 rows)

carpool=#
```

Figure 8.2: Notification trigger definition.

The notification execution evidence shows notification rows associated with booking events. The application exposes a notifications API to retrieve the current user's notifications and mark a notification as read.

```
carpool=# SELECT
    notification_id,
    user_id,
    booking_id,
    ride_id,
    title,
    notification_type,
    is_read
FROM notifications
ORDER BY notification_id DESC
LIMIT 10;
 notification_id | user_id | booking_id | ride_id |         title         | notification_type | is_read
-----------------+---------+------------+---------+-----------------------+-------------------+---------
               4 |       1 |         27 |       6 | Passenger cancelled   | booking_cancelled | f
               3 |       3 |         27 |       6 | Booking cancelled     | booking_cancelled | t
               2 |       1 |         27 |       6 | New passenger booking | new_booking       | t
               1 |       3 |         27 |       6 | Booking confirmed     | booking_confirmed | t
(4 rows)
```

Figure 8.3: Notification records generated by booking events.

8.3 Transactional booking integrity

The booking workflow uses a transaction and SELECT ... FOR UPDATE on the target ride. This serializes competing booking operations for the same ride, allowing the backend to validate available seats and update seat counts without accepting more seats than remain available. Cancellation similarly locks the ride before restoring seats and changing booking state.

9. FRONT END DEVELOPMENT

The frontend is implemented using HTML templates rendered by Flask, CSS for layout and visual styling, and vanilla JavaScript for browser-side interactions. The interface includes navigation for Dashboard, Find Rides, My Bookings, and Passenger Bookings. A logged-in driver can also book another driver's ride, so the My Bookings page is available to both account types.

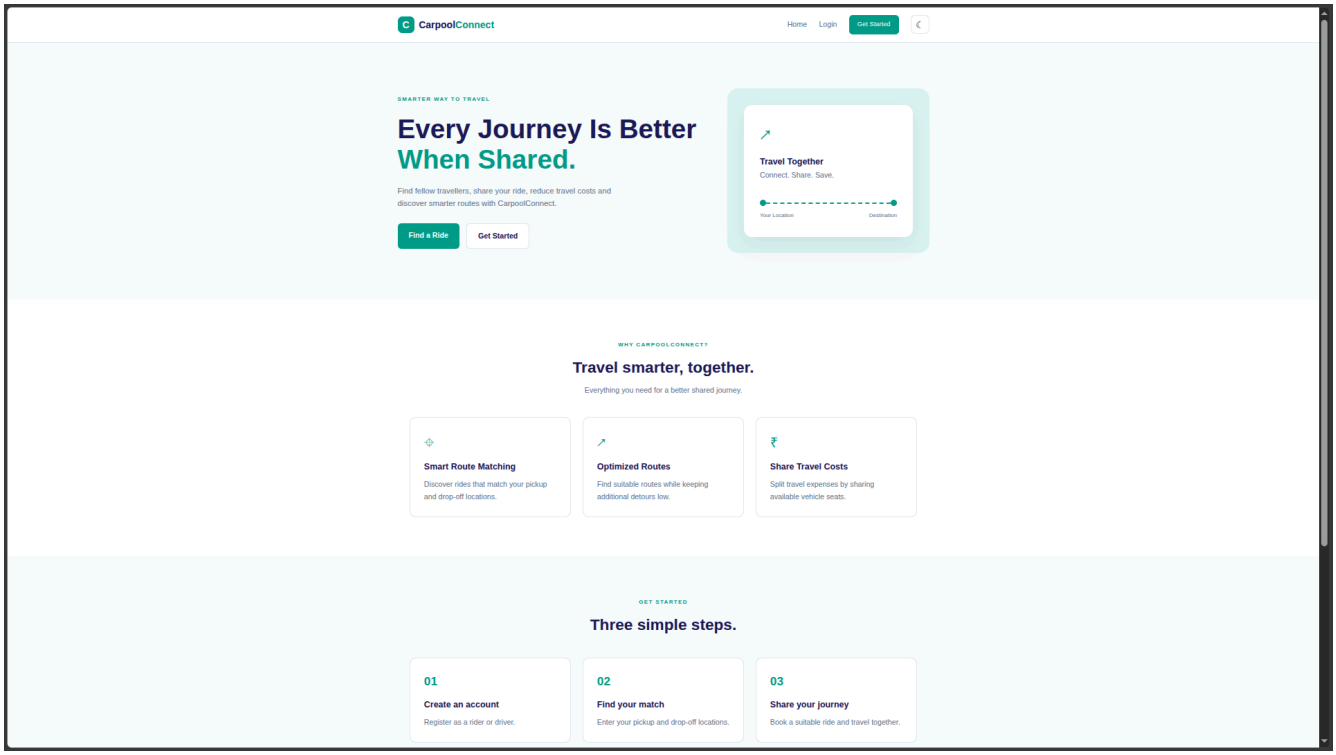


Figure 9.1: CarpoolConnect home page.

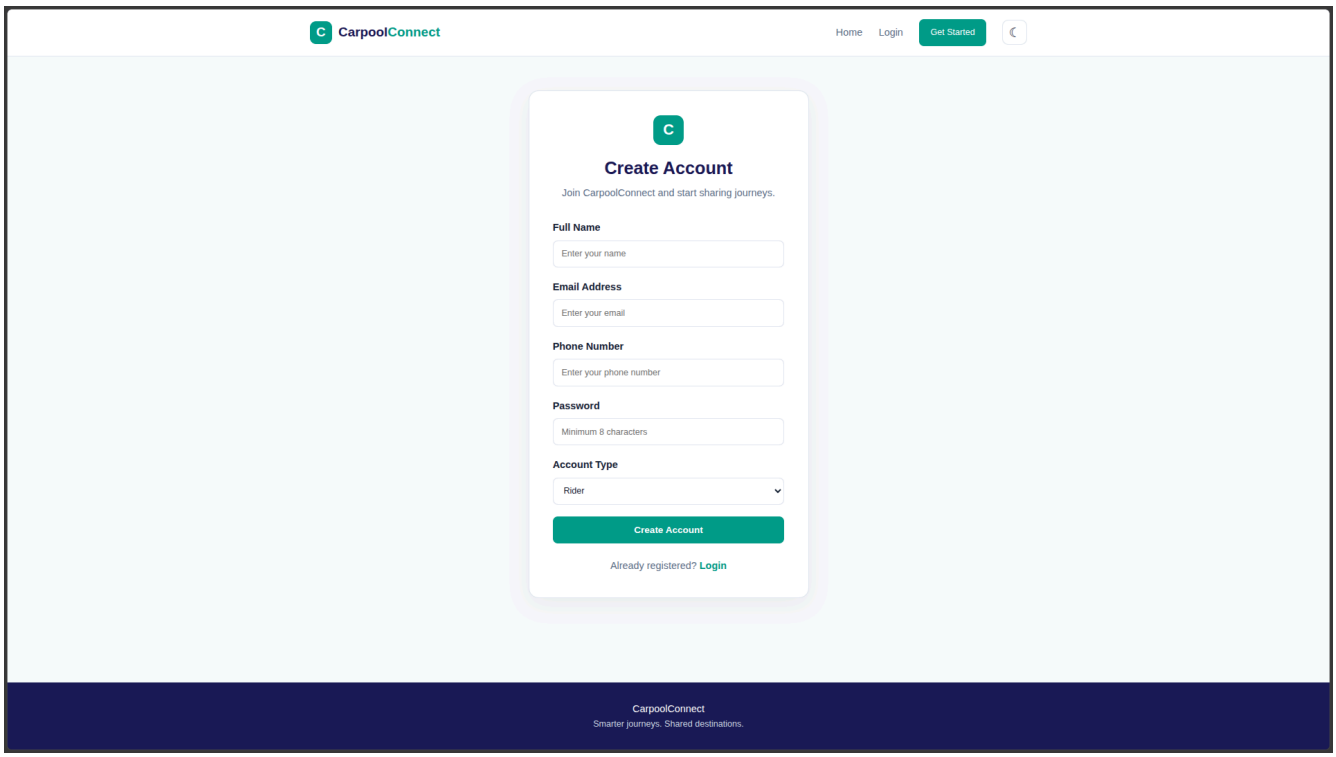


Figure 9.2: User registration page.

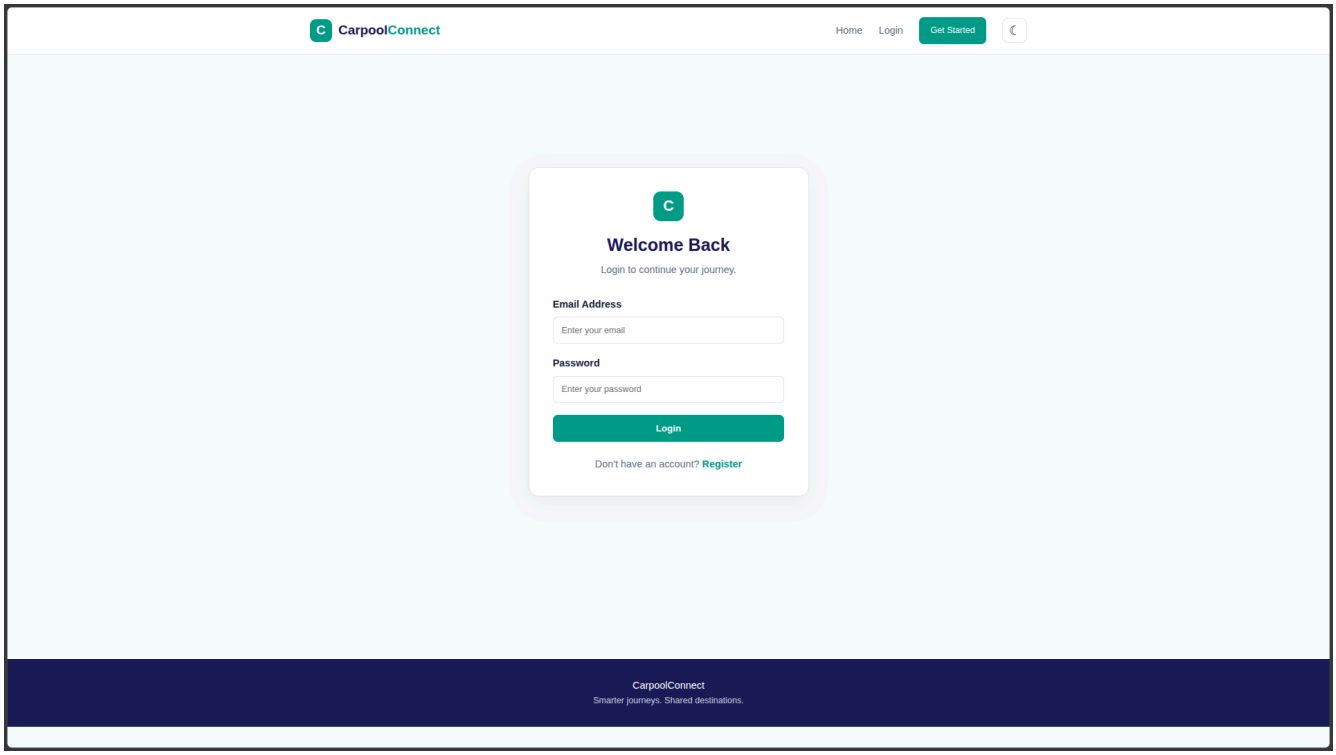


Figure 9.3: Login page.

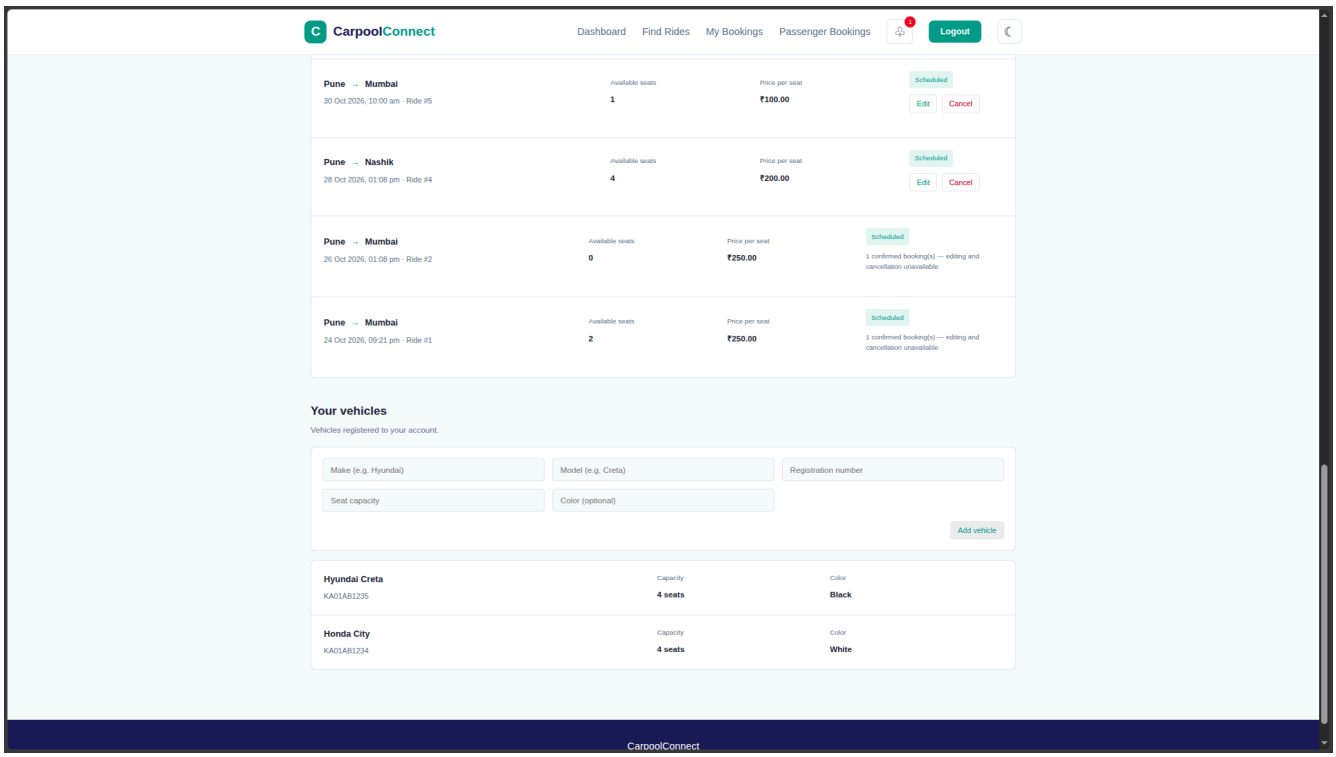


Figure 9.4: Driver dashboard, published rides, and registered vehicles.

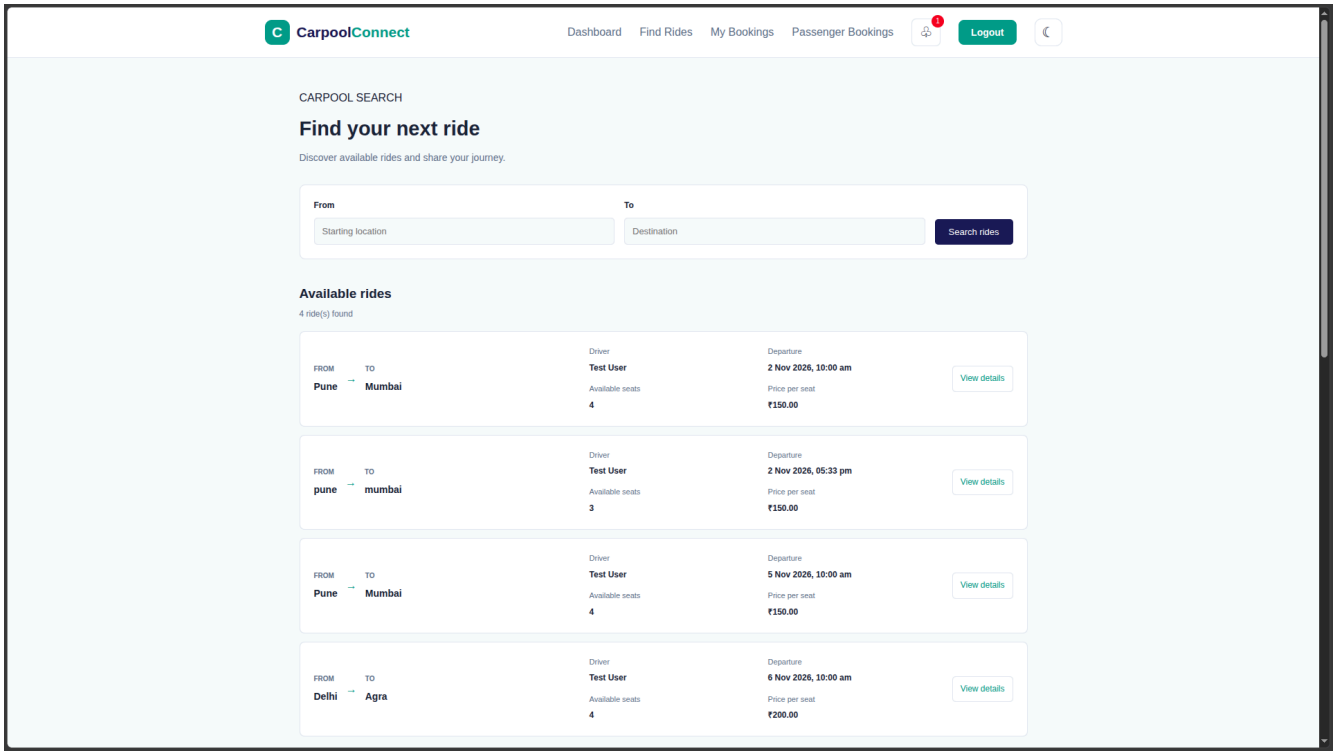


Figure 9.5: Find Rides page with available ride results.

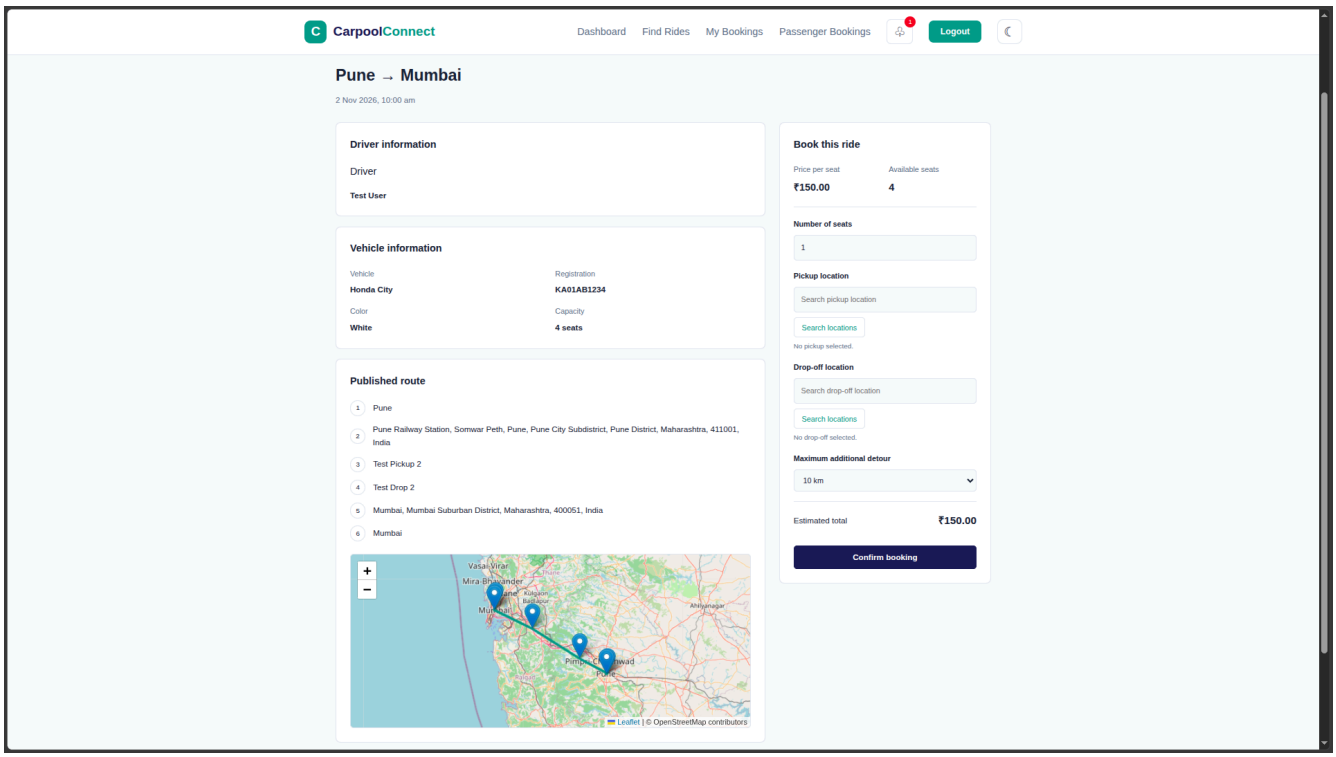


Figure 9.6: Ride details, route map, and booking interface.

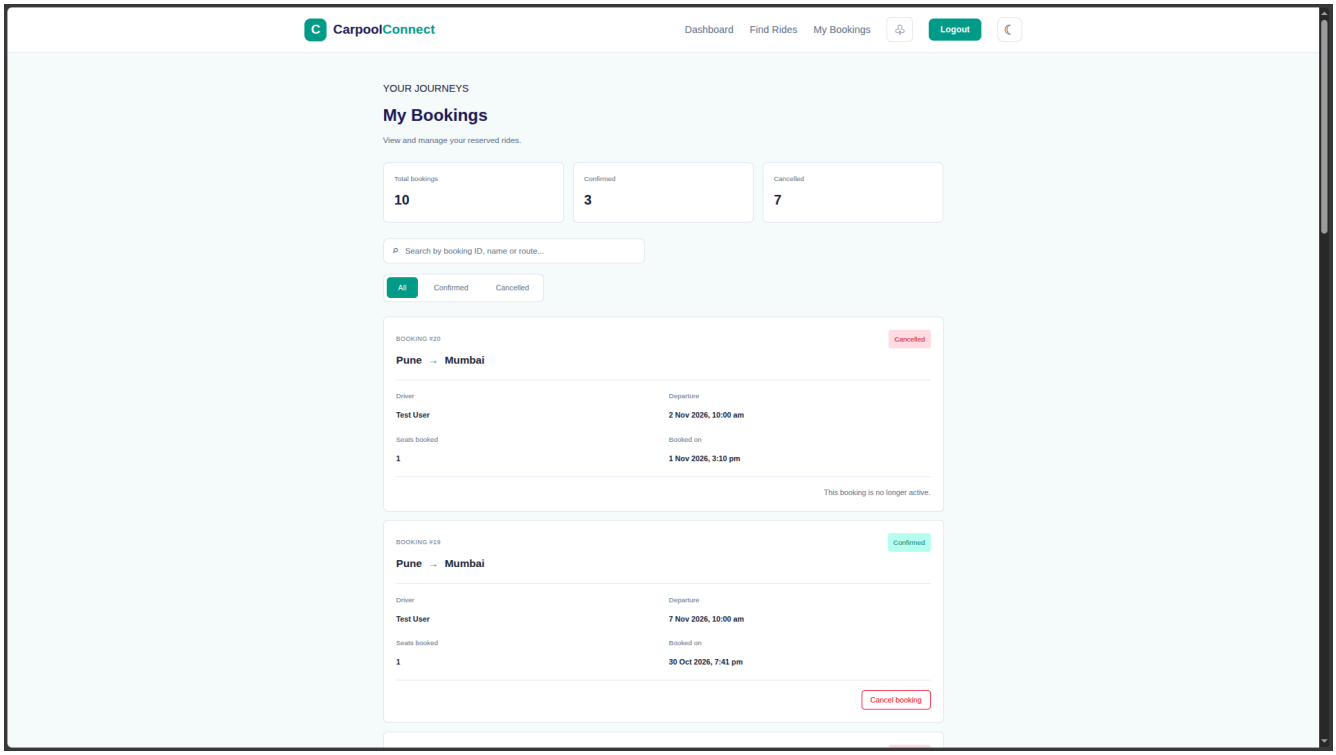


Figure 9.7: My Bookings page.

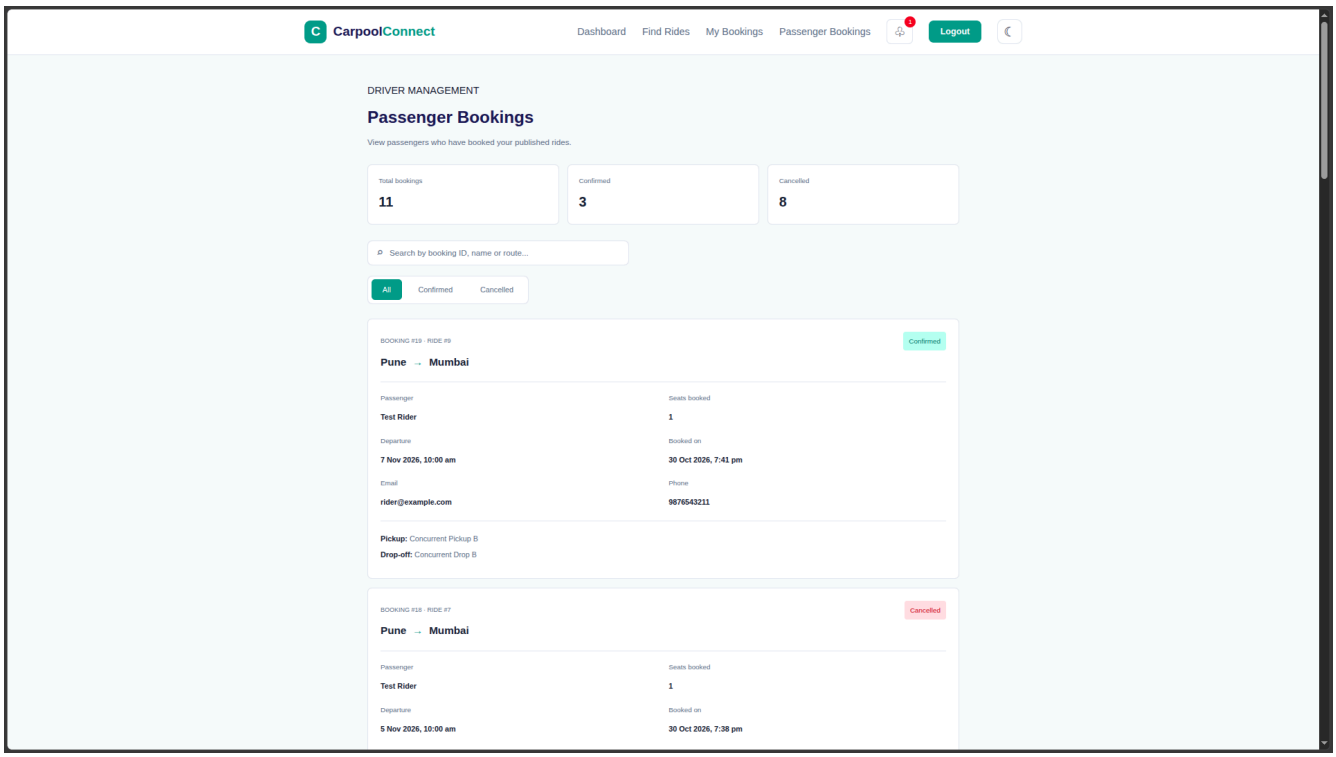


Figure 9.8: Passenger Bookings page for rides owned by the driver.

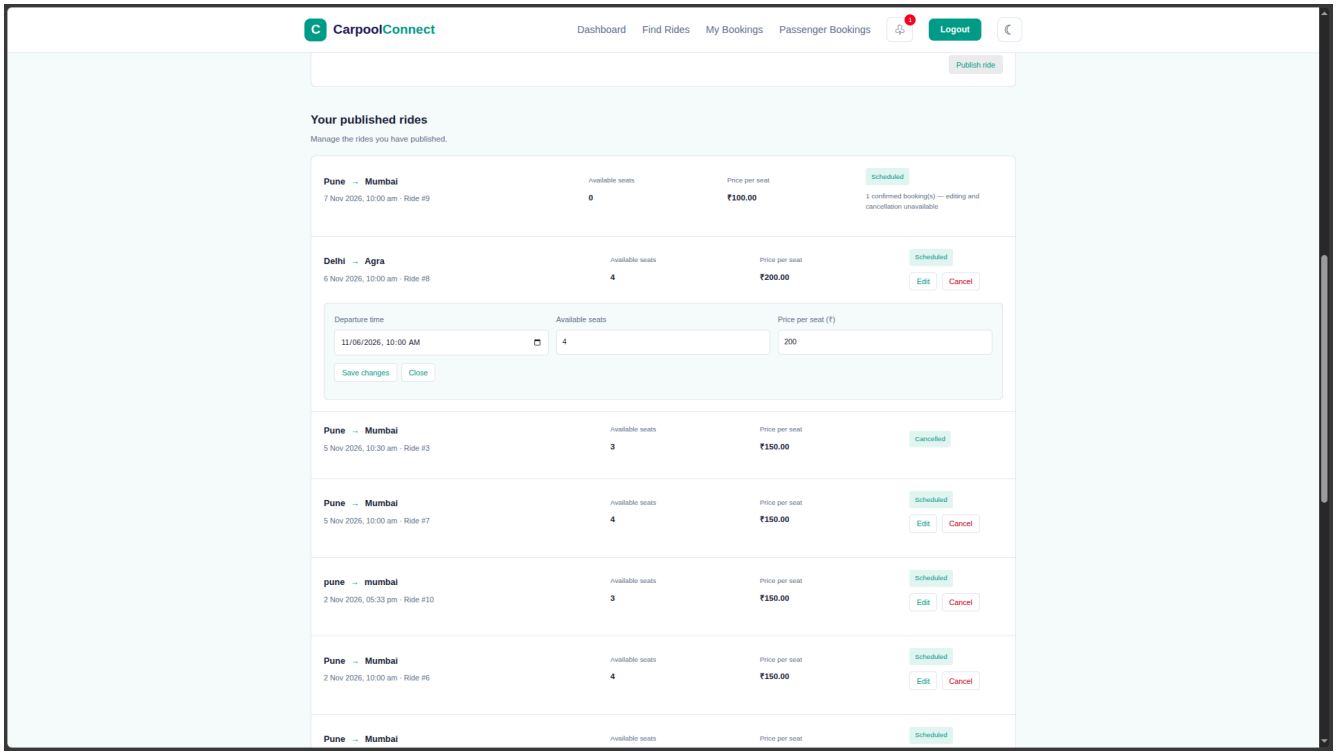


Figure 9.9: My Rides management interface.

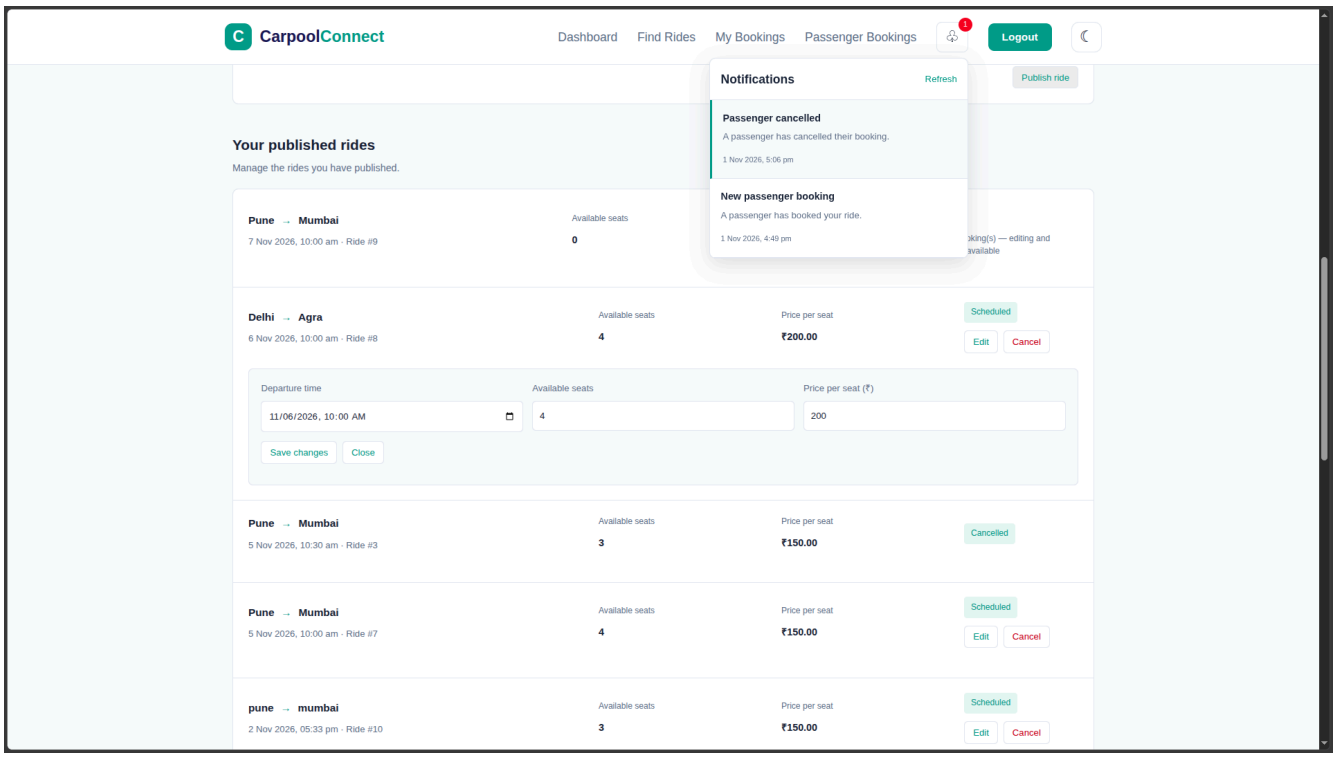


Figure 9.10: Notification dropdown with booking and cancellation events.

9.1 Main user workflows

Registration and login establish a session for the user.

Drivers register vehicles and publish rides from the dashboard.

Users search for rides using exact route details or nearby pickup/drop coordinates.

Users inspect ride details and submit booking requests.

Users manage their own reservations through My Bookings.

Drivers review passenger reservations through Passenger Bookings and manage eligible rides through My Rides.

Notifications appear in the navigation dropdown and can be marked as read.

A dark-mode screenshot was not included in the supplied image archive. The interface includes a theme toggle in the captured navigation, but this report does not include a separate dark-mode demonstration image.

REFERENCES

[1] PostgreSQL Global Development Group, “PostgreSQL Documentation.” *https://www.postgresql.org/docs/*

[2] pgvector, “Open-source vector similarity search for Postgres.” *https://github.com/pgvector/pgvector*

[3] Pallets Projects, “Flask Documentation.” *https://flask.palletsprojects.com/*

[4] Psycopg, “Psycopg 3 Documentation.” *https://www.psycopg.org/psycopg3/docs/*

[5] Leaflet, “Leaflet Documentation.” *https://leafletjs.com/reference.html*

[6] OpenStreetMap, “OpenStreetMap.” *https://www.openstreetmap.org/*

[7] CarpoolConnect project repository. *https://github.com/apatelpiyush/Carpooling-Coordination-with-Route-Optimization*

GitHub Link:

https://github.com/apatelpiyush/Carpooling-Coordination-with-Route-Optimization

APPENDIX A – PROJECT CONTRIBUTION

The project was developed collaboratively by Piyush A Patel and Ritesh Paruchuri. The following contribution summary is a proposed report-ready division based on the project modules and team discussions. The team should confirm or adjust the allocation before submission so that it accurately reflects individual work.

Work area	Piyush A Patel	Ritesh Paruchuri
Database design and PostgreSQL	Schema planning, database integration, constraints and indexing	Schema validation, data setup and database testing
Backend development	Flask application integration, authentication and API work	Ride and booking workflow implementation and testing
Route optimization	pgvector route embeddings and candidate ranking	Geographic route insertion, detour validation and matching tests
Frontend	Dashboard and ride-management integration	Ride search, booking pages and user-interface integration
Testing and documentation	API/security tests, architecture and database documentation	Functional/concurrency tests, evidence collection and user documentation

The above allocation must be verified by both team members. Replace any row that does not accurately describe the work performed.

APPENDIX B – ITEMS TO RECONCILE BEFORE FINAL SUBMISSION

- Remove the legacy RATINGS entity from the ER diagram, relational schema diagram, and SQL setup if Ratings is excluded from the final scope. The supplied diagrams and DDL screenshots still show RATINGS.

Add the notifications table, notify_booking_events() function, trigger, index, and required grants to the version-controlled SQL setup/migration. The development database contains these objects, but the supplied ZIP's SQL scripts do not fully reproduce them.

Add pickup_stop_inserted and drop_stop_inserted columns to the reproducible BOOKINGS DDL if the current application continues to use these fields.

Do not enable the legacy trg_booking_seats trigger alongside the current Python-managed seat decrement logic.

Capture the lower portion of notify_booking_events() if a complete function-definition screenshot is required by the evaluator. This report includes the available partial screenshot and describes its visible content only.

Refresh the table of contents and page numbers in Word after any final edits.